CloudSuite Multi-Tenant SaaS Platform

Project Documentation

Subject: System Design
Topic: Multi-Tenant SaaS Platform Design & Implementation
Technology: Python 3.8+, ReportLab, Standard Library

1. Problem Statement

CloudSuite is a multi-tenant SaaS platform similar to systems used by Salesforce and Zoho to provide software services to multiple organizations using a shared cloud infrastructure. The platform enables businesses to manage their operations such as customer management, reporting, analytics, and workflow automation through a web-based application. Each organization, known as a tenant, expects secure access to its own data, customized configurations, and reliable application performance.

As the platform scales to thousands of tenants and millions of users, the system must efficiently manage resource sharing while ensuring data isolation and security between tenants. The platform faces challenges such as uneven resource usage, tenant-specific customization handling, and maintaining consistent performance during peak workloads.

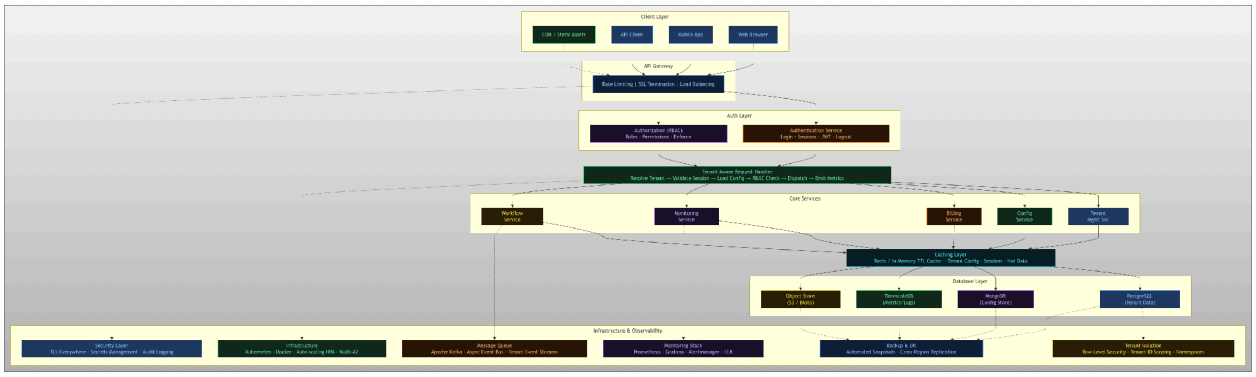
The system must support tenant onboarding, authentication, role-based access control, configuration management, billing, and monitoring. The architecture should support distributed services, tenant-aware databases, caching layers, and centralized monitoring systems.

2. Proposed Solution

CloudSuite is designed as a shared-infrastructure, isolated-data multi-tenant SaaS platform. All tenants share the same codebase and runtime environment, while data is isolated at the application layer using tenant-scoped queries and at the database layer using Row-Level Security (RLS). The system uses a service-oriented architecture with the following key design choices:

- Tenant-aware middleware that resolves tenant context on every request before any business logic executes.
- Cache-aside pattern using Redis (simulated with TTL dictionary) for tenant configurations to minimize DB round-trips.
- Role-Based Access Control (RBAC) with 5 hierarchical roles enforced at the dispatcher level.
- Plan-based resource limits (FREE / STARTER / PROFESSIONAL / ENTERPRISE) enforced at onboarding and runtime.
- Session-based authentication with tenant binding to prevent cross-tenant session reuse.
- Structured metric emission per request for per-tenant observability.

3. System Architecture



3.1 Layer Description

Layer	Components	Responsibility
Client Layer	Web Browser, Mobile App, API Client, CDN	User-facing access points
API Gateway	Rate Limiter, SSL Termination, Load Balancer	Entry point; enforces rate limits per tenant plan
Auth Layer	AuthenticationService, AuthorizationService	Session creation, RBAC enforcement
Middleware	TenantAwareRequestHandler	Resolves tenant, validates session, loads config, dispatches
Service Layer	Tenant, Config, Billing, Monitoring, Workflow	Core business logic, tenant-scoped
Cache Layer	Redis / TTL Cache	Tenant configs, session tokens, hot query results
Database Layer	PostgreSQL, MongoDB, TimescaleDB, Object Store	Persistent tenant-isolated data storage
Infra Layer	Kubernetes, Docker, Auto-scaling, Multi-AZ	Deployment, scaling, fault tolerance

4. Module Description

4.1 TenantManagementService

Handles full tenant lifecycle: onboarding, plan upgrades, suspension, and offboarding. On onboarding, creates the tenant record, sets resource limits per plan, creates the initial admin user, and generates the first billing record.

4.2 AuthenticationService

Tenant-scoped login and session management. Verifies credentials, enforces plan-level user caps, creates time-bounded sessions (8-hour TTL) with tenant_id bound to prevent cross-tenant reuse.

4.3 AuthorizationService (RBAC)

Enforces role-based permissions on every dispatch. Roles are hierarchical: SUPER_ADMIN > TENANT_ADMIN > MANAGER > MEMBER > VIEWER. Wildcard (*) grants all permissions to platform admins.

4.4 ConfigurationService

Loads tenant-specific configuration using a cache-aside pattern. On cache miss, reads from database, merges with plan-level defaults, and populates the TTL cache. Cache is invalidated on every config update.

4.5 TenantAwareRequestHandler

Core middleware pipeline: (1) Validate tenant active, (2) Validate session and tenant binding, (3) Load config, (4) Build request context, (5) Dispatch with RBAC, (6) Emit metrics. All errors are caught and returned as structured responses.

4.6 BillingService

Manages subscription records per tenant. Creates billing records on onboarding, tracks plan and amount, and supports marking invoices as paid. In production, integrates with Stripe for payment processing.

4.7 MonitoringService

Records structured per-tenant metric events (request latency, error counts) and provides aggregate summaries. In production, exports to Prometheus with Grafana dashboards.

4.8 InMemoryDatabase

Simulates a relational database with tenant-aware data isolation. Every data table is keyed by tenant_id. Cross-tenant queries are structurally impossible. Maps to PostgreSQL with Row-Level Security in production.

4.9 TenantConfigCache

TTL-based in-memory cache (300-second default). Implements cache-aside: get → miss → load from DB → set → return. Maps to Redis in production with tenant-prefixed keys.

5. Database Design

The database layer uses a polyglot persistence strategy: different data types are stored in the most appropriate database engine. Tenant isolation is enforced via a tenant_id column on every table, with PostgreSQL Row-Level Security (RLS) policies ensuring queries can only access rows belonging to the authenticated tenant.

5.1 Core Tables (PostgreSQL)

Table	Key Columns	Purpose
tenants	tenant_id (PK), name, domain, plan, status, created_at	Tenant registry
users	user_id (PK), tenant_id (FK), email, role, password_hash, is_active	User accounts per tenant
sessions	session_id (PK), user_id (FK), tenant_id, role, issued_at, expires_at	Active sessions
tenant_data	row_id, tenant_id, table_name, data (JSONB), created_by, created_at	Tenant business records
billing_records	record_id (PK), tenant_id, plan, amount_usd, period_start, period_end, status	Billing history
metric_events	event_id, tenant_id, service, metric_name, value, timestamp, tags (JSONB)	Monitoring data

5.2 Configuration Store (MongoDB)

Tenant configurations are stored in MongoDB as flexible JSON documents, allowing each tenant to have different configuration keys without schema migrations. Document key: tenant_id. Fields include: theme, timezone, language, features (array), custom settings.

5.3 Metrics Store (TimescaleDB)

Time-series metric data (request latencies, error counts, API usage) is stored in TimescaleDB (PostgreSQL extension for time-series). Tables are partitioned by time and tenant_id for efficient range queries and retention policies.

5.4 Row-Level Security Policy (Production)

```
-- Enable RLS on tenant data table
ALTER TABLE tenant_data ENABLE ROW LEVEL SECURITY;

-- Policy: each session can only see its own tenant's rows
CREATE POLICY tenant_isolation ON tenant_data
  USING (tenant_id = current_setting('app.current_tenant_id'));

-- Set tenant context per DB connection
SET app.current_tenant_id = 'tenant_xxxx';
```

6. Requirements Analysis (Q1)

6.1 Functional Requirements

#	Requirement	Justification
FR 1	Tenant Onboarding & Management	Platform must register new organizations with their own namespace, admin user, and plan configuration before any services can be used.
FR 2	User Authentication	Secure identity verification is the first line of defense; without it, any user could access any tenant's data.
FR 3	Role-Based Access Control	Different users within a tenant have different authority levels; RBAC prevents privilege escalation.
FR 4	Tenant-Specific Configuration	Each tenant may need different themes, features, and settings; hardcoded configs would make customization impossible.
FR 5	Multi-Tenant Data Management	Core business operations (CRM, reporting, workflows) require persistent, isolated data storage per tenant.
FR 6	Billing & Subscription Management	SaaS platforms monetize through subscriptions; billing tracks plan, usage, and payment status.
FR 7	Monitoring & Alerting	Operators need visibility into per-tenant performance, error rates, and resource consumption.
FR 8	Tenant Suspension / Offboarding	Non-paying or policy-violating tenants must be suspended without affecting other tenants.

6.2 Non-Functional Requirements

#	Requirement	Target	Justification
NF R1	Scalability	10,000+ tenants	Platform must grow without architectural changes.
NF R2	Availability	99.9% SLA	Downtime directly impacts all tenants revenue.
NF R3	Performance	<200ms p95 latency	Slow responses erode user trust and retention.
NF R4	Security/Isolation	Zero cross-tenant leakage	Data breaches destroy reputation and incur legal liability.
NF R5	Fault Tolerance	No single point of failure	Service outages cascade to all tenants if not isolated.
NF R6	Compliance	GDPR, SOC2 ready	Enterprise customers require regulatory compliance.
NF R7	Maintainability	Modular services	Independent deployability reduces blast radius of changes.

7. System Architecture Design (Q2)

The CloudSuite architecture follows a layered, service-oriented design. A client request flows through the API Gateway, which enforces rate limits per tenant plan and terminates SSL. The request then hits the Authentication Service to validate the session token. The Authorization Service checks RBAC permissions for the requested action.

The Tenant-Aware Request Handler is the central middleware that ties all services together. It resolves the tenant from the request, validates the session belongs to that tenant, loads the tenant's configuration (from cache or DB), builds a request context, enforces RBAC via the Authorization Service, dispatches to the appropriate core service, and emits metrics.

Core services (Tenant Management, Billing, Workflow, Monitoring) are stateless and communicate with the database layer via tenant-scoped queries. An async event bus (Kafka) decouples long-running operations (email sends, report generation) from the synchronous request path.

8. Tenant Isolation and Resource Management (Q3)

8.1 Data Isolation Strategy

- Every database table includes a tenant_id column. All queries include WHERE tenant_id = :tid.
- PostgreSQL Row-Level Security (RLS) policies enforce isolation at the database engine level, ensuring even a buggy query cannot leak data.
- Sessions carry tenant_id. The request handler verifies session.tenant_id == request.tenant_id before any processing.
- The in-memory cache uses tenant_id as the key prefix, preventing config bleed-over between tenants.
- Separate Kubernetes namespaces can be used for Enterprise tenants requiring full process isolation.

8.2 Resource Sharing and Limits

All tenants share the same compute nodes and database clusters (shared-infrastructure model), which maximizes hardware utilization. Resource fairness is enforced through:

Plan	Max Users	Storage	API Rate Limit	Monthly Price
Free	5	1 GB	100 req/min	$0
Starter	25	10 GB	500 req/min	$29
Professional	100	100 GB	2,000 req/min	$99
Enterprise	9,999	5 TB	10,000 req/min	$499

9. Algorithm and Implementation (Q5)

9.1 Tenant-Aware Request Handling Pipeline

The core algorithm in TenantAwareRequestHandler.handle() implements a security-first pipeline where every guard must pass before the next step executes. If any step fails, an appropriate error is returned immediately without executing business logic.

```
def handle(tenant_id, session_id, action, payload):
    # Step 1: Resolve and validate tenant
    tenant = db.get_tenant(tenant_id)
    if not tenant or tenant.status != ACTIVE: raise Error

    # Step 2: Validate session and tenant binding
    session = auth_svc.validate_session(session_id)
    if session.tenant_id != tenant_id: raise ForbiddenError

    # Step 3: Load tenant-specific config (cache-aside)
    config = config_svc.load_config(tenant_id)

    # Step 4: Build immutable request context
    ctx = RequestContext(tenant_id, session, config)

    # Step 5: Dispatch with RBAC enforcement
    result = dispatch(ctx, action, payload)

    # Step 6: Emit telemetry metric
    monitor_svc.record_metric(tenant_id, 'latency', elapsed_ms())
    return {success: True, data: result}
```

9.2 Cache-Aside Config Loading

```
def load_config(tenant_id):
    # 1. Check TTL cache (O(1) lookup)
    cached = config_cache.get(tenant_id)
    if cached: return cached          # Cache HIT

    # 2. Cache MISS -> load from database
    config = db.get_config(tenant_id)

    # 3. Merge with plan-level defaults
    merged = merge_with_defaults(config, tenant.plan)

    # 4. Populate cache with TTL=300s
    config_cache.set(tenant_id, merged)
    return merged
```

10. Scalability and Fault Tolerance (Q6)

10.1 Horizontal Scaling

- All services are stateless containers, allowing Kubernetes Horizontal Pod Autoscaler (HPA) to add replicas during load spikes.
- The API Gateway distributes traffic across service replicas using round-robin or least-connections load balancing.
- Read replicas handle reporting and analytics queries, reducing load on the primary database.
- Tenant configs cached in Redis reduce database load by 80-90% for high-frequency config lookups.

10.2 Fault Tolerance

- Multi-AZ deployment: each service runs in at least 2 availability zones; AZ failure causes automatic failover.
- Database primary-replica replication with automatic promotion on primary failure (RTO < 60s).
- Kafka message queue decouples async work; failed jobs are retried with exponential backoff.
- Circuit breaker pattern prevents cascading failures when downstream services are degraded.
- Health checks at container and load balancer levels automatically remove unhealthy pods from traffic rotation.

10.3 Real-World Parallels

Salesforce uses a similar shared-infrastructure model with tenant_id-scoped queries and org-level governor limits (analogous to our plan resource limits). Zoho employs a microservices architecture with per-module scaling, similar to CloudSuite's service-oriented design. Both use Redis for session and config caching, and Kafka for async event processing.

11. Technology Stack

Layer	Technology	Reason
Language	Python 3.8+	Readable, extensive ecosystem, fast prototyping
API Gateway	Kong / AWS API Gateway	Built-in rate limiting, auth plugins, SSL offload
Auth	JWT + bcrypt	Stateless tokens, industry standard, secure hashing
Cache	Redis	Sub-millisecond latency, TTL support, cluster mode
Primary DB	PostgreSQL + RLS	ACID compliance, native JSON, Row-Level Security
Config Store	MongoDB	Flexible schema for varied tenant configurations
Metrics DB	TimescaleDB	Time-series optimized, PostgreSQL compatible
Message Queue	Apache Kafka	High-throughput, durable, tenant-partitioned topics
Container Orch.	Kubernetes	Auto-scaling, self-healing, multi-AZ deployments
Monitoring	Prometheus + Grafana	Per-tenant dashboards, alerting, SLA tracking
CI/CD	GitHub Actions	Automated testing, Docker build, Kubernetes deploy

12. Future Scope

- SSO / SAML 2.0 integration for Enterprise tenants using identity providers like Okta or Azure AD.
- AI-powered anomaly detection on per-tenant metrics to proactively identify security threats or performance degradation.
- Tenant self-service portal for onboarding, billing management, and configuration updates without admin intervention.
- Multi-region deployment with data residency controls (GDPR: EU data stays in EU).
- GraphQL API layer for flexible client queries with tenant-aware query depth limiting.
- Usage-based billing model (pay per API call / GB stored) in addition to flat subscription plans.
- Automated tenant data export and deletion for GDPR right-to-erasure compliance.
- A/B testing framework for rolling out platform features to a subset of tenants.